

WallyGuard2

Low-level loop architecture

A sequential Python controller coordinates four LLM roles through Chia and Ray. Agents propose investigations, tests, reviews and RTL fixes. The controller independently executes tests and decides whether a patch qualifies for export.

Execution topology

Component	Placement	Responsibility
Controller	Ray job driver	Iteration state, ordering, verification decisions and archival.
Model execution	opencode_creds worker	OpenCode sessions, model-provider calls and MCP tool requests.
Simulation / tools	wally_sim worker	Checkout access, shell commands, compilation, Spike, Verilator and evidence.

remote() submits a Chia task and waits for its result. The four agent roles run sequentially within one iteration. A named LLMCapacity actor additionally limits model sessions; the default capacity is one.

Iteration flow



Main success path. Invalid tests and rejected claims exit or enter bounded repair loops. main() creates an iteration record and calls run_attempt(). No-bug outcomes skip the Critic and Fixer. A claimed bug reaches the Critic only after the controller validates the positive control and reproduces the same baseline mismatch.

Primary entry points: loop.py — main(), run_attempt(), remote(), agent_stage().

Agent calls and workspace

One agent call, end to end

1. `agent_stage()` snapshots protected source and test inputs.
2. `ask_agent()` constructs a role-specific prompt and starts a temporary MCP ToolServer on the simulation worker.
3. The model call acquires an LLM capacity lease. OpenCode runs on the model worker and sends HTTP/MCP requests to the tool server.
4. Tester commands start in the run directory; other roles start in the checkout. \$WALLY always identifies the checkout. The response is parsed against the role's JSON schema.
5. The server stops, resources are released, and file-integrity checks run. Stage duration and diagnostics are retained.

Tool	Behavior
<code>run_command</code>	Returns completion or <code>RUNNING/job_id</code> within about two seconds. One active command per tool instance; 120s default deadline, overridable for builds.
<code>command_status</code>	Waits up to 30 seconds; polling does not cancel the command.
<code>validate_reproducer</code>	Checks the saved execution contract. A ready result is not evidence of an RTL bug.

Full command output is stored in logs; the model receives a bounded tail. Tool shutdown cancels owned commands and process cleanup handles descendants. Rate-limit retries and MCP recovery are bounded.

Persistent workspace and run state

Location	Purpose
<code>cvw/</code>	Original checkout; controller-managed experiments use the copy.
<code>wally-worktrees/wally-shared/</code>	Reusable working checkout with dependencies and build assets.
<code>wally-shared.json</code> / <code>.lock</code>	Saved baseline and ownership metadata; preparation lock.
<code>runs/<tag>/</code>	Test sources, binaries, logs, <code>attempt.json</code> , snapshots and diagnostics.
<code>candidate-bugs/</code> / <code>confirmed-bugs/</code>	Exported patches, separated by verification scope.

The first iteration seeds the workspace. Later iterations restore tracked files to the baseline while retaining build data. Active or incompletely archived attempts, unresolved source conflicts, and a changed source HEAD block reuse. Test artifacts live directly in the run directory.

Simulation tools are resolved on the simulation host. `WALLY_SPIKE` pins the absolute oracle path; its RISC-V simulator identity is checked before agents start and before baseline/fix test execution.

Sources: `orchestration/tool_runtime.py`, `toolchain.py`, `context.py`, `artifact_guard.py`; `loop.py` — `make_worktree()`, `ask_agent()`, `save_attempt()`.

Verification and reliability

Evidence required before fixing

The controller fingerprints and archives the reproducer inputs, validates the contract, builds the programs, and runs a positive control plus the suspected failing test. Wally and Spike must execute the same ELF. The Tester may return a reference to its saved contract. The native self-check path requires explicit completion and expected/actual evidence; signature mode compares independently produced signatures.

A baseline mismatch must repeat with the same fingerprint (two total runs by default). A Critic then reviews the claim. After the Fixer changes RTL, the controller reruns the reproducer and enabled regressions, requests a second review, and verifies that original inputs and the reviewed RTL remain unchanged.

Export	Required gates
Candidate	Reproducible baseline mismatch; approved bug review; passing targeted fix; approved fix review, with broader verification not fully enabled.
Confirmed	All candidate gates plus passing operator-selected directed regression and full regression.

Build failures, oracle failures, invalid tests, host timeouts and DUT watchdog failures are separate outcomes. None establishes an RTL mismatch. Critic approval cannot override failed verification. WALLY_RUN_REGRESSION defaults to 0; full confirmation also requires WALLY_DIRECTED_COMMAND.

Integrity and recovery

Source and test snapshots bracket agent stages. Generated output trees are pruned before input traversal. The Fixer may edit existing RTL .sv files; the original reproducer remains frozen. Unauthorized edits are archived and restored. These are integrity checks around shell execution, not an operating-system sandbox. Controller event logs are excluded from the reproducer fingerprint.

Defaults: up to two test repairs after the initial attempt, three fix attempts, one MCP recovery retry, and four additional rate-limit retries. attempt.json is updated atomically. Evidence is retained on rejected, partial and failed attempts.

Performance and observability

Agent sessions, command execution, baseline repeats and reviews add serially to elapsed time. Workspace reuse, bounded prompts and waiting polls reduce overhead. Stage timings, command durations, tool-metrics.json, events.jsonl and Chia profiles expose where time is spent. The Architect receives the last 40 outcomes with observed failure reasons and brief unverified notes; full evidence stays on disk. There is no model-weight training.

Sources: orchestration/test_preflight.py, result_classifier.py, verification_state.py, processes.py, stage_runner.py; loop.py — test_fingerprint(), verify_command(), export_patch().

Scope: this document describes the inspected local implementation, including the 20 September simulator-selection correction. It does not claim an end-to-end throughput improvement or a completed full hardware regression.